Laboratoire 08 — Raspberry Pi 5, Linux, Python et mes découvertes en IA

Matisse Rhéaume-Viale

8 novembre 2026



Techniques de génie électronique programmable

Laboratoire 08 : Raspberry Pi 5, Linux et Python

Rapport de laboratoire et approfondissement sur l'IA locale

Cours : 243-315-LI INTRODUCTION AUX SYSTÈMES D'EXPLOITATION
Enseignant : Rin-Élysé Baudelet
Groupe : 0001

Table des matières

1. Objectif du travail **4**

2. Matériel et logiciels utilisés **4**

3. Préparation du Raspberry Pi **5**
3.1 Choix du système d'exploitation . . . 5
3.2 Pourquoi ne pas utiliser directement HDMI ? . . . 5

4. Premier problème : Raspberry Pi introuvable sur Ethernet **5**
4.1 Branchement initial . . . 5
4.2 Premier test de communication . . . 6
4.3 Vérification de l'adaptateur Ethernet . . . 6
6.1 Pourquoi utiliser DHCP ? . . . 7
6.3 Configuration de l'adresse du Mac . . . 7
6.4 Démarrage du serveur DHCP . . . 7

9. Activation du bureau à distance avec VNC **9**
Résultat . . . 9

10. Activation du Wi-Fi du Raspberry Pi **10**
10.1 Premier problème Wi-Fi . . . 10
10.2 Recherche des réseaux Wi-Fi . . . 10

11. Configuration d'eduroam en ligne de commande **10**
12.1 Configuration permanente du Raspberry Pi . . . 11
12.2 Configuration permanente du Mac . . . 12
12.3 Vérification des adresses . . . 12
12.4 Nouvel essai au cégep le 2 novembre 2026 . . . 12

13. Configuration réseau obtenue le 27 octobre **12**

14. Résultats des essais du 27 octobre **13**

15. Bilan des manipulations du 27 octobre **13**

16. À la maison : configuration de mon Raspberry Pi 5 personnel **14**
16.1 Assemblage et premiers essais . . . 14
16.2 Configuration de l'accès à distance avec TigerVNC . . . 14
16.3 Identification et vérification du système d'exploitation . . . 14
16.4 Choix des identifiants et configuration du compte utilisateur . . . 15

17. Découverte physique du Raspberry Pi 5 **15**

18. Python et essais supplémentaires **16**
18.1 Mon idée : récupérer des informations sur les Pi utilisés en classe . . . 16
18.2 Un premier essai d'enregistrement dans SQLite . . . 16
18.3 Ce que j'ai appris avec cette expérience . . . 17

19. Sources et références **17**

20. Journal complémentaire de mes expériences et de mes recherches sur les Raspberry Pi 5 **18**
20.1 Les Raspberry Pi 5 dont j'ai parlé ne sont pas tous le même appareil . . . 18
20.2 Ce que j'ai réussi le 27 octobre : un vrai réseau direct entre mon Mac et le Pi . . . 19
20.3 Le 2 novembre : pourquoi j'ai gardé aussi les échecs dans le rapport . . . 19
20.4 Le Raspberry Pi 5 personnel : assemblage, démarrage et choix de l'accès à distance . . . 19
20.5 Vérification de la version de Linux installée . . . 19
20.6 Mes premières vraies mesures de ressources système . . . 20

20.7 Mon idée de récupérer les adresses MAC des Pi du magasin TGE . . . 21
20.8 Tests envisagés entre deux Raspberry Pi 5 . . . 21
20.9 Le serveur que je voudrais pouvoir démarrer et arrêter quand je veux . . . 21

21. Les notions techniques que j'ai voulu approfondir 21
21.1 Correspondance avec les exigences du laboratoire . . . 22

22. Architecture du Raspberry Pi 5 et comparaison avec un PC 22
22.1 Un ordinateur monocarte, pas un microcontrôleur . . . 22
22.2 Schéma matériel conceptuel . . . 23
22.3 Inventaire reproductible de l'appareil . . . 23

23. Linux : du démarrage aux processus 23
23.1 Chaîne de démarrage . . . 23
23.2 Mémoire virtuelle, cache et swap . . . 24
23.3 Diagnostic réseau par couches . . . 24

24. Python sur Raspberry Pi OS 24
24.1 Installation et environnement virtuel . . . 24
24.2 Premier programme mesurable . . . 24

25. Introduction à l'intelligence artificielle locale 25
25.1 Différencier IA, apprentissage automatique et LLM . . . 25
25.2 Entraînement et inférence . . . 25

26. Comment un LLM génère du texte 25
26.1 Tokenisation . . . 25
26.2 Embeddings . . . 25
26.3 Attention causale . . . 26
26.4 Attention multi-têtes et couches profondes . . . 26
26.5 Logits, température et échantillonnage . . . 27

27. Mathématiques de l'entraînement 27
27.1 Produit matriciel et paramètres . . . 27
27.2 Perte d'entropie croisée . . . 27
27.3 Descente de gradient . . . 27
27.4 Perplexité . . . 27

28. Quantification des LLM 28
28.1 Pourquoi réduire la précision ? . . . 28
28.2 Exemple pour un modèle de 3 milliards de paramètres . . . 28
28.3 Quantification affine . . . 28
28.4 GGUF et types de quantification . . . 28

29. Mémoire, KV cache et performances du Raspberry Pi 5 29
29.1 Estimation du KV cache . . . 29
29.2 Mémoire totale approximative . . . 29
29.3 Débit théorique limité par la mémoire . . . 29
29.4 Temps de génération . . . 29

30. Protocole de benchmark reproductible des modèles locaux 29
30.1 Objectif . . . 29
30.2 Informations à relever . . . 30
30.3 Exemple de procédure avec llama.cpp . . . 30
30.4 Tableau de résultats à remplir après exécution . . . 30

31. Classification, débruitage et modèles de diffusion 30
31.1 Classification . . . 30

31.2 Débruitage de signaux et d'images 31
31.3 Modèles de diffusion 31
31.4 Expériences possibles sur Pi 5 32

32. Limites, sécurité, sources complémentaires et travaux futurs **32**
32.1 Sécurité des réseaux et des modèles 32
32.2 Ce qui reste à faire 32
32.3 Sources techniques complémentaires 32

Note sur l'utilisation de l'intelligence artificielle **33**
Pourquoi j'ai voulu garder ces notes dans le rapport 33

Ce rapport contient autant ce que j'ai fait en classe que les manipulations que j'ai poursuivies avec mon Raspberry Pi 5 personnel. J'ai gardé mes réussites, mes problèmes et les solutions que j'ai essayées, parce que ce sont aussi ces démarches qui m'ont permis de comprendre le système. À partir de la section 20, j'ai ajouté un journal d'expérimentation et des approfondissements sur Linux et l'IA locale. Je distingue ce que j'ai réellement mesuré de ce que j'ai seulement recherché ou prévu.

Quand un petit pictogramme ChatGPT avec un point d'exclamation apparaît dans le rapport, il indique un passage où je précise comment une IA m'a aidé. Je ne considère pas les réponses d'une IA comme des preuves : je les compare à mes observations, aux outils comme `man` et à la documentation technique.

1. Objectif du travail

Le but principal du laboratoire 08 était de découvrir le Raspberry Pi 5, son fonctionnement et son système d'exploitation basé sur Debian. On devait aussi apprendre à reconnaître les composants de la carte, documenter l'installation du système et utiliser Python. Les GPIO font également partie des possibilités à explorer pour relier le Pi à des composants électroniques.

De mon côté, je me suis donné quelques objectifs supplémentaires. Comme j'utilise déjà mon MacBook Pro en classe et que je n'aime pas avoir un autre écran qui prend de la place sur mon bureau, je voulais au départ me servir de mon Mac pour afficher le Raspberry Pi et le contrôler à distance. Je voulais éviter d'avoir à ajouter un clavier, une souris et un deuxième écran juste pour travailler sur le Pi.

Je voulais également comprendre comment relier directement les deux appareils avec un câble Ethernet, même sans routeur ni connexion Internet sur ce lien. Ensuite, je voulais que le Pi conserve Internet par le Wi-Fi du cégep tout en restant accessible par Ethernet depuis mon Mac.

C'est comme ça que j'ai fini par toucher à plusieurs notions qu'on voyait dans le cours : les adresses IP, DHCP, les interfaces réseau et SSH, mais aussi des outils que j'ai découverts pendant mes essais, comme `tcpdump`, `dnsmasq`, NetworkManager et VNC. J'ai pris beaucoup de notes pendant mes manipulations, y compris lorsque quelque chose ne fonctionnait pas du premier coup.

2. Matériel et logiciels utilisés

Voici le matériel et les logiciels que j'ai utilisés au laboratoire ou pendant mes essais sur mon Pi personnel. Ils n'ont pas tous été utilisés en même temps.

Équipement ou logiciel	Utilisation
Raspberry Pi 5	Ordinateur Linux à configurer
Raspberry Pi OS Desktop	Système d'exploitation du Pi
Carte microSD	Stockage du système
MacBook Pro	Ordinateur principal et écran à distance
Adaptateur AX88179A	Connexion Ethernet du Mac
Câble Ethernet	Liaison directe Mac–Pi
Raspberry Pi Imager 2.0.12	Installation du système
Terminal macOS	Commandes réseau et SSH
Homebrew	Installation de dnsmasq
dnsmasq 2.93	Serveur DHCP temporaire
TigerVNC / WayVNC	Affichage du bureau du Pi
NetworkManager / nmcli	Configuration réseau Linux
eduroam	Accès Internet du cégep

3. Préparation du Raspberry Pi

3.1 Choix du système d'exploitation

Pour le laboratoire, on devait installer **Raspberry Pi OS**, la distribution basée sur Debian qui est adaptée au Raspberry Pi. C'est ce que j'ai utilisé depuis le début. C'est le système que j'ai utilisé dès le départ, conformément aux consignes du laboratoire.

J'ai utilisé Raspberry Pi Imager pour préparer la carte microSD, en suivant la documentation officielle de Raspberry Pi. Comme je voulais accéder au bureau graphique du Pi à distance, j'ai installé une version avec environnement de bureau.

Lors de la préparation, j'ai configuré :

- Nom d'hôte : `etudiant`
- Utilisateur Linux : `etudiant`
- Mot de passe : défini dans Imager
- SSH : activé avec authentification par mot de passe
- Connexion Wi-Fi : non configurée initialement

J'ai activé SSH avant le premier démarrage afin de pouvoir essayer de rejoindre le Raspberry Pi depuis mon Mac sans être obligé de brancher immédiatement un écran sur le Pi.

3.2 Pourquoi ne pas utiliser directement HDMI ?

Au début du laboratoire, je voulais vraiment utiliser mon MacBook comme écran. J'avais pensé brancher le Pi au port HDMI de mon Mac, mais ce port sert à **sortir** une image vers un autre écran, pas à recevoir directement le signal vidéo du Raspberry Pi.

Une carte de capture HDMI vers USB aurait été une possibilité, mais j'aurais dû utiliser du matériel supplémentaire. J'ai donc choisi d'essayer SSH pour le terminal et VNC pour voir et contrôler le bureau graphique.

Utilisation de ChatGPT — Quand j'ai cherché comment afficher le bureau sur mon Mac, ChatGPT m'a proposé TigerVNC. C'était intéressant parce que j'avais déjà utilisé ce logiciel dans un projet personnel avec son aide. J'ai ensuite suivi les réglages et vérifié que ça fonctionnait sur ma propre configuration. Voir aussi la documentation officielle sur l'accès à distance de Raspberry Pi.

Je précise que **je n'ai pas évité HDMI pendant toutes mes expérimentations**. Plus tard, lorsque je travaillais sur un script pour récupérer des informations de Pi utilisés par des étudiants, j'ai branché un Raspberry Pi à un écran de la classe avec un câble **micro-HDMI**. Dans cette situation, l'accès direct à l'écran était plus pratique pour obtenir le terminal et vérifier les informations de l'appareil.

4. Premier problème : Raspberry Pi introuvable sur Ethernet

4.1 Branchement initial

J'ai branché directement le Raspberry Pi au Mac avec un câble Ethernet.

Sur macOS, l'adaptateur était reconnu sous le nom `AX88179A`, associé à l'interface `en9`. Le lien physique indiquait une vitesse de 1 Gbit/s.

Par contre, le Mac avait obtenu une adresse IP automatique :

```
169.254.30.16/16
```

Cette adresse appartient à la plage IPv4 *link-local* (169.254.0.0/16). Elle est utilisée lorsqu'un appareil n'obtient pas d'adresse IPv4 par DHCP et qu'un mécanisme d'auto-adressage est disponible.

En effet, aucun routeur ni serveur DHCP ne distribuait d'adresses IP entre les deux ordinateurs. **Utilisation de ChatGPT** — Dans le cours, j'avais surtout travaillé avec des adresses en `/24`. Quand j'ai vu une adresse en `/16`, je voulais comprendre la différence, plutôt que de simplement recopier une explication. J'ai donc demandé à ChatGPT de m'expliquer le principe du masque et du préfixe CIDR, puis de me poser un petit quiz pour vérifier

que j'avais compris. Le `/16` indique 16 bits de préfixe réseau, alors que `/24` en indique 24. Les adresses automatiques IPv4 `169.254.0.0/16` sont décrites dans la RFC 3927.

4.2 Premier test de communication

Dans le Terminal macOS, j'ai exécuté :

```
ping -c 4 etudiant.local
```

Résultat :

```
ping: cannot resolve etudiant.local: Unknown host
```

Le Mac n'arrivait pas à résoudre le nom d'hôte du Raspberry Pi.

J'ai ensuite tenté d'utiliser SSH :

```
ssh etudiant@etudiant.local
```

La connexion n'a pas abouti.

Cela ne prouvait pas que SSH était désactivé. Le Mac ne parvenait pas à trouver le Raspberry Pi par son nom, et je n'avais pas encore trouvé d'adresse IP qui me permettait de le joindre directement. Ces premières commandes de diagnostic, comme `ping` et `ssh`, étaient liées à des notions que j'avais déjà vues dans le cours d'introduction aux systèmes d'exploitation. J'ai donc commencé avec les outils que je connaissais avant de chercher d'autres moyens de comprendre le problème.

4.3 Vérification de l'adaptateur Ethernet

J'ai utilisé :

```
ifconfig en9 | grep -E 'status:|inet '
```

Résultat :

```
inet 169.254.30.16
status: active
```

L'interface était active, mais sa configuration IP n'était pas suffisante pour garantir une communication avec le Pi. La vérification de l'interface Ethernet avec `ifconfig` s'inscrivait aussi dans ce qu'on avait appris en classe sur les interfaces et l'adressage réseau. Par la suite, j'ai utilisé des outils supplémentaires pour analyser les échanges plus en détail. # 5. Diagnostic réseau avec tcpdump

Pour savoir si le Raspberry Pi envoyait réellement du trafic sur le câble, j'ai utilisé `tcpdump`.

Commande exécutée sur le Mac :

```
sudo tcpdump -ni en9 \
  'udp port 67 or udp port 68 or arp or udp port 5353'
```

Cette commande permet de surveiller :

- UDP 67 et 68 : échanges DHCP
- ARP : recherche d'adresses physiques sur le réseau local
- UDP 5353 : mDNS, utilisé notamment pour les noms `.local`

J'ai ensuite redémarré le Raspberry Pi tout en laissant la capture réseau active.

Dans les résultats, j'ai trouvé une requête provenant de l'adresse MAC :

`98:FE:54:3C:94:8E`

Extrait :

```
0.0.0.0.68 > 255.255.255.255.67
BOOTP/DHCP Request from 98:fe:54:3c:94:8e
```

Cette observation était importante.

Elle démontrait que le Raspberry Pi avait démarré suffisamment pour initialiser son interface Ethernet et tenter d'obtenir une configuration réseau.

Le problème ne se situait donc pas uniquement au niveau du câble. Il manquait aussi une méthode d'attribution des adresses IP. Pour mieux comprendre ce que je voyais, j'ai comparé mes observations avec le fonctionnement de DHCP décrit dans la RFC 2131. Cette référence explique notamment les échanges utilisés pour obtenir une configuration IP. # 6. Installation et configuration d'un serveur DHCP

6.1 Pourquoi utiliser DHCP ?

DHCP signifie *Dynamic Host Configuration Protocol*.

Son rôle principal est de fournir automatiquement des paramètres réseau aux appareils, notamment :

- Une adresse IP
- Un masque de sous-réseau
- Une passerelle par défaut, si nécessaire
- Des serveurs DNS, si nécessaires

Normalement, c'est un routeur qui offre ce service.

Puisque mon Mac était connecté directement au Raspberry Pi, sans routeur, j'ai décidé de configurer temporairement mon Mac comme serveur DHCP. Je connaissais déjà le rôle général de DHCP grâce au cours d'introduction aux systèmes d'exploitation. Mon défi était surtout de réussir à utiliser cette notion dans une situation un peu différente : une liaison directe Mac–Pi, sans routeur qui distribue automatiquement les adresses IP. ## 6.2 Installation de dnsmasq

J'avais déjà Homebrew sur le Mac.

J'ai vérifié sa présence avec :

```
command -v brew
```

Résultat :

```
/opt/homebrew/bin/brew
```

J'ai ensuite installé dnsmasq :

```
brew install dnsmasq
```

Dnsmasq est un logiciel léger qui peut notamment fournir des services DHCP et DNS sur un réseau local (voir la documentation officielle de dnsmasq).

6.3 Configuration de l'adresse du Mac

J'ai attribué au Mac l'adresse locale `192.168.77.1` sur son interface Ethernet :

```
sudo ifconfig en9 inet 192.168.77.1 \
  netmask 255.255.255.0 alias
```

Cette commande ajoutait temporairement une adresse IPv4 sur `en9`.

J'ai choisi le réseau privé `192.168.77.0/24`, indépendant du réseau Wi-Fi du cégep.

6.4 Démarrage du serveur DHCP

J'ai lancé dnsmasq avec la configuration suivante :

```
sudo "$(brew --prefix)/opt/dnsmasq/sbin/dnsmasq" \
  --no-daemon \
  --conf-file=/dev/null \
  --port=0 \
```

```
  --interface=en9 \
  --dhcp-range=192.168.77.2,192.168.77.20,255.255.255.0,1h \
  --dhcp-host=98:fe:54:3c:94:8e,192.168.77.2 \
  --dhcp-option=option:router \
  --dhcp-option=option:dns-server \
  --log-dhcp
```

Explication des principaux paramètres :

Paramètre	Fonction
`--no-daemon`	Garde dnsmasq au premier plan
`--conf-file=/dev/null`	Ignore la configuration par défaut
`--port=0`	Désactive la fonction serveur DNS
`--interface=en9`	Utilise l'interface Ethernet du Mac
`--dhcp-range`	Définit la plage d'adresses disponibles
`--dhcp-host`	Réserve une adresse à l'adresse MAC du Pi
`--dhcp-option`	Évite d'annoncer une passerelle ou un DNS inutile
`--log-dhcp`	Affiche les échanges DHCP

Le serveur a démarré correctement :

```
dnsmasq: started, version 2.93 DNS disabled
dnsmasq-dhcp: DHCP, IP range
192.168.77.2 -- 192.168.77.20
```

Utilisation de ChatGPT — Pour construire la commande `dnsmasq`, j'ai d'abord consulté les pages de manuel avec `man` et les options d'aide comme `--help`. J'ai ensuite demandé à ChatGPT de relever les erreurs possibles et de m'expliquer certaines options. Je ne me suis pas fié uniquement à sa réponse : j'ai relu la commande et vérifié mes résultats avant de conclure qu'elle fonctionnait. **Analyse assistée par un modèle d'IA local** — Pendant l'observation du trafic, j'avais beaucoup de texte qui défilait dans le terminal. J'ai utilisé un modèle de langage local, accessible sur une infrastructure de collaboration distincte du Raspberry Pi, pour m'aider à repérer les passages intéressants dans cette quantité de données. Le modèle, l'infrastructure et les méthodes utilisées sont couverts par une entente de confidentialité ; je ne peux donc pas fournir ces détails ni les fichiers correspondants. Les échanges DHCP que je présente dans le rapport sont ceux que j'ai retenus pour expliquer le fonctionnement observé. # 7. Attribution réussie de l'adresse IP

Après avoir redémarré le Raspberry Pi, j'ai observé une séquence DHCP complète :

```
DHCPDISCOVER(en9) 98:fe:54:3c:94:8e
DHCPOFFER(en9) 192.168.77.2
DHCPREQUEST(en9) 192.168.77.2
DHCPACK(en9) 192.168.77.2
```

Les quatre étapes signifient :

1. **DHCPDISCOVER :** le Raspberry Pi recherche un serveur DHCP.
2. **DHCPOFFER :** le Mac lui propose l'adresse `192.168.77.2`.
3. **DHCPREQUEST :** le Raspberry Pi demande cette adresse.
4. **DHCPACK :** le Mac confirme l'attribution.

Le serveur recevait aussi le nom d'hôte `etudiant`, ce qui confirmait que la requête provenait de la configuration attendue.

C'était le premier résultat décisif du dépannage : le serveur DHCP et le Raspberry Pi arrivaient enfin à communiquer pour attribuer une adresse IP. **Utilisation d'une IA locale** — Le modèle m'a aussi aidé à organiser l'information retenue et à distinguer les événements utiles à mon analyse. Comme cette partie du travail est confidentielle, je n'entre pas dans les détails techniques du traitement. En revanche, je comprends les quatre messages

DHCP présentés ici et je peux expliquer leur rôle avec les notions vues en cours. Je n'attribue pas cette analyse au Raspberry Pi lui-même : l'outil d'IA fonctionnait dans un autre environnement. # 8. Connexion au Raspberry Pi avec SSH

Une fois l'adresse attribuée, j'ai testé la communication depuis le Mac :

```
ping -c 4 192.168.77.2
```

Ensuite, j'ai établi la connexion SSH :

```
ssh etudiant@192.168.77.2
```

Cette fois, la connexion a réussi.

SSH signifie *Secure Shell*. Il permet d'ouvrir une session de terminal distante sur un autre ordinateur à travers le réseau, avec une connexion chiffrée.

Grâce à SSH, je pouvais maintenant exécuter les commandes Linux du Raspberry Pi directement depuis le Terminal de mon Mac.

Cette étape était importante puisque je n'avais pas besoin d'un écran physique pour continuer sa configuration.

9. Activation du bureau à distance avec VNC

SSH permet de contrôler le terminal, mais je voulais aussi accéder à toute l'interface graphique de Raspberry Pi OS.

J'ai donc configuré VNC, qui signifie *Virtual Network Computing*. La documentation Raspberry Pi sur l'accès à distance explique la configuration de SSH et VNC.

Sur le Raspberry Pi, j'ai lancé :

```
sudo raspi-config
```

Dans le menu de configuration, j'ai utilisé les options suivantes :

- **System Options → Boot / Auto Login → Desktop Autologin**
- **Interface Options → VNC → Enable**

Le démarrage automatique du bureau était particulièrement utile puisque le Raspberry Pi n'avait aucun moniteur HDMI connecté.

Sur le Mac, j'ai utilisé TigerVNC pour me connecter à :

`192.168.77.2`

Le Raspberry Pi utilisait WayVNC comme serveur VNC, tandis que TigerVNC servait de client sur le Mac.

Résultat

J'ai finalement obtenu le bureau complet de Raspberry Pi OS dans une fenêtre de mon MacBook.

Je pouvais utiliser :

- Le bureau Linux
- Le clavier et le trackpad du Mac
- Le terminal du Raspberry Pi
- Chromium
- Les applications graphiques du système

J'ai même pu ouvrir ChatGPT dans Chromium à partir du bureau du Raspberry Pi, tout en regardant ce bureau sur mon Mac avec TigerVNC. Je trouve quand même ça assez drôle que mon premier réflexe, pour vérifier le navigateur et Internet sur mon Raspberry Pi, ait été d'ouvrir ChatGPT. On dirait que même pour tester une connexion, il faut absolument que je vérifie si mon assistant IA fonctionne. Au moins, ça m'a permis de confirmer à la fois l'accès à Chromium et l'affichage du bureau par VNC.

Cela confirmait que la solution fonctionnait sans écran HDMI supplémentaire.

10. Activation du Wi-Fi du Raspberry Pi

Je voulais ensuite que le Raspberry Pi ait son propre accès à Internet par le Wi-Fi du cégep, plutôt que de dépendre du partage de connexion que j'avais utilisé à partir de mon **MacBook**. Je précise que je n'ai pas utilisé les données cellulaires LTE de mon téléphone pour cette étape.

10.1 Premier problème Wi-Fi

J'ai commencé par vérifier les interfaces :

```
nmcli device status
```

Résultat initial :

```
eth0    ethernet  connected
wlan0   wifi      unavailable
```

L'Ethernet fonctionnait, mais le Wi-Fi était indisponible.

J'ai utilisé les commandes suivantes pour activer l'interface :

```
sudo nmcli radio wifi on
sudo rfkill unblock wifi
sudo nmcli device set wlan0 managed yes
```

J'ai également vérifié la configuration du pays Wi-Fi, qui devait être réglée sur le Canada (`CA`).

Après ces manipulations :

```
wlan0   wifi   disconnected
```

Cette fois, l'interface était disponible, mais pas encore connectée à un réseau.

10.2 Recherche des réseaux Wi-Fi

J'ai exécuté :

```
nmcli -f IN-USE,SSID,SIGNAL,SECURITY \
  device wifi list ifname wlan0
```

Le Raspberry Pi détectait plusieurs réseaux, dont :

SSID	Sécurité
`climoilou`	WPA2 802.1X
`climoilou_visiteur`	Réseau ouvert
`climoilou_objet_connecte`	WPA2
`eduroam`	WPA2 802.1X

J'ai choisi eduroam, le réseau sécurisé utilisé dans les établissements d'enseignement et aussi en raison de reglement de sécuirté et legal, Chat GPT aurais un peu plus de permisions sur Eduroam que Climoilou.

11. Configuration d'eduroam en ligne de commande

Le réseau eduroam nécessite une authentification plus complexe qu'un réseau résidentiel WPA2 utilisant un simple mot de passe partagé.

D'après le guide de connexion Wi-Fi du Cégep Limoilou, la configuration utilise :

— Réseau : `eduroam`

— Méthode EAP : PEAP
— Authentification interne : MSCHAPV2
— Certificats CA : certificats système
— Domaine : `cegeplimoilou.ca`
— Identifiant : numéro de DA suivi de `@cegeplimoilou.ca`

J'ai adapté ces paramètres pour NetworkManager sur Raspberry Pi OS.

Les commandes de configuration étaient :

```
read -rp "Numero de DA : " DA
```

Puis :

```
sudo nmcli connection add \
  type wifi \
  con-name eduroam \
  ifname wlan0 \
  ssid eduroam \
  wifi-sec.key-mgmt wpa-eap \
  802-1x.eap peap \
  802-1x.phase2-auth mschapv2 \
  802-1x.identity "${DA}@cegeplimoilou.ca" \
  802-1x.system-ca-certs yes \
  802-1x.domain-suffix-match cegeplimoilou.ca
```

J'ai ensuite activé la connexion :

```
sudo nmcli --ask connection up eduroam
```

L'option `--ask` permet de saisir les informations d'authentification de manière interactive, plutôt que de placer le mot de passe directement dans la commande.

La connexion à eduroam a réussi. **Utilisation de ChatGPT** — Avant de passer par ces commandes, j'avais essayé de configurer eduroam directement dans l'interface graphique du Raspberry Pi. Le problème, c'est que l'interface prenait beaucoup trop de temps à répondre et ça rendait les essais pénibles. J'ai donc demandé à ChatGPT de m'aider à traduire les paramètres de connexion en commandes `nmcli`. J'ai ensuite vérifié moi-même que l'authentification fonctionnait et que le Pi pouvait accéder à Internet. # 12. Utilisation simultanée d'Ethernet et du Wi-Fi

Une fois le Wi-Fi fonctionnel, je voulais que les deux interfaces aient des fonctions distinctes.

— Ethernet : communication locale entre le Mac et le Pi.
— Wi-Fi : connexion Internet du Pi.

J'ai donc remplacé la configuration DHCP temporaire de l'Ethernet par des adresses IP statiques.

12.1 Configuration permanente du Raspberry Pi

Sur le Pi :

```
sudo nmcli connection modify "Wired connection 1" \
  ipv4.method manual \
  ipv4.addresses "192.168.77.2/24" \
  ipv4.gateway "" \
  ipv4.dns "" \
  ipv4.never-default yes \
  connection.autoconnect yes
```

La propriété `ipv4.never-default yes` est particulièrement importante.

Elle empêche NetworkManager d'utiliser cette connexion Ethernet comme route Internet par défaut.

Cela permet au Pi de garder son Ethernet pour le Mac et de choisir le Wi-Fi pour atteindre Internet.

12.2 Configuration permanente du Mac

Sur macOS :

```
sudo networksetup -setmanual "AX88179A" \
  192.168.77.1 255.255.255.0 ""
```

Le Mac utilise maintenant une adresse fixe sur son adaptateur Ethernet, sans passerelle sur cette connexion.

12.3 Vérification des adresses

Après le redémarrage, j'ai exécuté sur le Pi :

```
ip -4 addr show eth0
```

Résultat :

```
inet 192.168.77.2/24
valid_lft forever preferred_lft forever
```

Puis :

```
ip route get 1.1.1.1
```

Résultat :

```
1.1.1.1 via 10.50.0.1 dev wlan0
src 10.50.8.225
```

Cela confirme que :

- `eth0` utilise `192.168.77.2/24`.
- La route vers Internet passe par `wlan0`.
- La passerelle Wi-Fi est `10.50.0.1`.
- L'adresse Wi-Fi observée était `10.50.8.225`.

J'ai ensuite arrêté dnsmasq sur le Mac et testé de nouveau SSH.

La connexion fonctionnait toujours.

Le serveur DHCP temporaire n'était donc plus nécessaire pour maintenir la liaison Ethernet.

12.4 Nouvel essai au cégep le 2 novembre 2026

Lors d'un nouvel essai le 2 novembre, j'ai tenté de me connecter à `192.168.77.2` comme avant. Le `ping` ne répondait pas, l'entrée ARP indiquait `(incomplete)` et SSH affichait `Host is down`.

Je me suis ensuite rendu compte d'un détail important : **je n'utilisais plus le Raspberry Pi 5 de ma professeure, mais mon Raspberry Pi 5 personnel**. L'adresse IP fixe `192.168.77.2` avait été configurée et validée sur le premier appareil. Je ne pouvais donc pas supposer que mon nouveau Pi possédait automatiquement cette même adresse ou le même profil Ethernet.

Ça expliquait pourquoi mes tentatives en reprenant simplement l'ancienne adresse n'étaient pas suffisantes. Les messages d'erreur ne prouvaient pas que le câble ou SSH étaient défectueux. Il me restait à identifier la configuration réseau du Pi personnel, puis à établir une nouvelle liaison avec mon Mac.

13. Configuration réseau obtenue le 27 octobre

Voici l'architecture obtenue :

```
MacBook Pro
   |
   | AX88179A / en9
   | 192.168.77.1/24
```

```
   |
   | Câble Ethernet
   |
   | eth0
   | 192.168.77.2/24
Raspberry Pi 5
   |
   | wlan0
   | 10.50.8.225
   |
   | Wi-Fi eduroam
   |
   | Passerelle 10.50.0.1
   |
Internet
```

Les deux interfaces peuvent fonctionner simultanément parce qu'elles appartiennent à deux réseaux différents.

Le réseau `192.168.77.0/24` sert exclusivement aux communications locales avec le Mac.

La connexion Wi-Fi institutionnelle utilise l'adresse `10.50.8.225` dans le réseau du cégep. Je n'ai pas indiqué de masque pour ce réseau, puisque je n'ai pas conservé sa valeur dans les résultats présentés.

La route par défaut utilise le Wi-Fi.

14. Résultats des essais du 27 octobre

Test	Résultat
Détection de l'adaptateur Ethernet du Mac	Réussi
Capture du trafic DHCP avec tcpdump	Réussi
Attribution DHCP avec dnsmasq	Réussi
Accès SSH au Raspberry Pi	Réussi
Affichage graphique avec TigerVNC	Réussi
Activation de l'interface Wi-Fi	Réussi
Authentification à eduroam	Réussi
Accès Internet depuis le Pi	Réussi
Adresse IPv4 fixe sur le Pi	Réussi
Route Internet par wlan0	Réussi
Connexion SSH sans dnsmasq	Réussi

J'ai également envisagé de créer des raccourcis pour ouvrir automatiquement SSH et VNC, ainsi que d'utiliser SCP pour transférer des fichiers entre les appareils. Ces fonctions ont été proposées, mais elles n'ont pas encore été validées par des essais documentés dans cette séance.

15. Bilan des manipulations du 27 octobre

Ce travail m'a permis de configurer et de dépanner un Raspberry Pi 5 sans utiliser de moniteur externe.

Le problème initial venait principalement de l'absence d'une configuration IP fonctionnelle sur la liaison Ethernet directe. Même si les deux appareils étaient physiquement connectés, ils ne pouvaient pas communiquer correctement.

L'utilisation de tcpdump m'a permis de voir que le Raspberry Pi envoyait des requêtes DHCP. J'ai donc installé dnsmasq sur le Mac pour lui attribuer temporairement une adresse IP.

Une fois la communication établie, j'ai pu accéder au Pi par SSH et activer son bureau à distance avec VNC.

J'ai ensuite configuré son Wi-Fi avec NetworkManager et l'authentification 802.1X d'eduroam. Finalement, j'ai donné des adresses statiques à la liaison Ethernet, ce qui a éliminé le besoin d'un serveur DHCP permanent.

Ce laboratoire m'a permis de mieux comprendre la différence entre une connexion physique et une connexion réseau fonctionnelle, le rôle de DHCP, la configuration des interfaces Linux et le fonctionnement de l'accès à distance.

À la fin de cette séance, je pouvais utiliser le Raspberry Pi 5 depuis mon MacBook, avec un bureau graphique, un accès SSH et une connexion Internet par le Wi-Fi du cégep. Je devais toutefois vérifier que la liaison Ethernet fonctionnerait encore lors des séances suivantes.

---

16. À la maison : configuration de mon Raspberry Pi 5 personnel

16.1 Assemblage et premiers essais

Puisque je me suis acheté moi-même un Raspberry Pi 5, j'ai recommencé les étapes de préparation à la maison. Cette fois, je n'ai pas refait exactement les mêmes manipulations, car j'avais déjà compris plusieurs problèmes rencontrés au laboratoire.

Après avoir suivi les instructions d'assemblage et de préparation du manuel papier fourni, j'ai réussi à afficher le bureau de mon Pi 5 sur un écran.

Puisque je préfère utiliser mon Raspberry Pi 5 avec mon MacBook Pro, j'ai immédiatement procédé à ces étapes :

— Activer Raspberry Pi Connect
— Me connecter au shell de mon Pi 5 à distance.
— Faire des tests.

16.2 Configuration de l'accès à distance avec TigerVNC

Au départ, j'utilisais Raspberry Pi Connect pour accéder à l'environnement graphique de mon Raspberry Pi 5 depuis mon MacBook Pro. Cependant, j'ai rencontré plusieurs problèmes avec cette méthode, notamment des ralentissements importants, des images figées et parfois un écran noir.

J'ai donc décidé d'essayer une autre solution : **TigerVNC**, un logiciel qui permet de contrôler graphiquement un ordinateur à distance à partir d'un réseau local.

Pour établir la connexion, j'ai configuré l'accès VNC sur mon Raspberry Pi, puis j'ai utilisé la commande `hostname -I` pour connaître son adresse IP locale. J'ai obtenu l'adresse `10.0.0.57`, que j'ai ensuite entrée dans TigerVNC Viewer sur mon Mac.

La connexion a fonctionné et j'ai réussi à accéder au bureau de mon Raspberry Pi directement depuis mon MacBook Pro. Cette méthode me permet d'utiliser l'écran, le clavier et la souris de mon Mac sans devoir brancher un moniteur externe sur le Raspberry Pi.

Sources :

— Raspberry Pi — Remote Access (VNC)
— TigerVNC — Site officiel

16.3 Identification et vérification du système d'exploitation

Comme mon Raspberry Pi 5 était livré avec une carte microSD contenant déjà un système d'exploitation, je voulais vérifier quelle distribution Linux était installée avant de commencer les laboratoires.

J'ai utilisé la commande `cat /etc/os-release` dans le terminal pour obtenir les informations sur le système.

Voici les résultats obtenus :

Information	Résultat
Distribution détectée	Debian GNU/Linux

Information	Résultat
Version majeure	13 (`13.7`)
Nom de code	Trixie
Architecture	ARM 64 bits
Noyau Linux	6.18.50+rpt-rpi-2712

J'ai également exécuté les commandes `uname -m`, `getconf LONG_BIT` et `uname -r` afin de vérifier l'architecture et la version du noyau Linux.

Les résultats `aarch64` et `64` ont confirmé que mon système fonctionne en 64 bits.

J'ai appris que Raspberry Pi OS est une distribution basée sur Debian, adaptée aux ordinateurs Raspberry Pi. Mes résultats montrent une base Debian 13 Trixie et un noyau pour Raspberry Pi. Par contre, `cat /etc/os-release` ne permet pas à lui seul de savoir exactement quelle image avait été installée sur ma carte microSD avant que je la reçoive.

Comme le système préinstallé fonctionnait pour mes premiers essais, j'ai décidé de le conserver pour le moment plutôt que de reformater la carte microSD.

Sources :

— Raspberry Pi — Documentation Raspberry Pi OS
— Raspberry Pi — Versions officielles des systèmes d'exploitation
— Debian — Informations sur Debian 13 Trixie

16.4 Choix des identifiants et configuration du compte utilisateur

Dans les consignes du laboratoire, il est demandé de configurer le Raspberry Pi avec les identifiants suivants :

— **Nom d'utilisateur :** `etudiant`
— **Mot de passe :** mot de passe commun fourni dans les consignes du laboratoire.

Cependant, comme j'utilise mon Raspberry Pi 5 personnel plutôt qu'un appareil appartenant au cégep, j'ai décidé de conserver mes propres identifiants.

Mon nom d'utilisateur est `matisserv` et j'utilise un mot de passe personnel différent de celui demandé dans le laboratoire.

J'ai choisi cette configuration pour éviter d'avoir à modifier mes paramètres personnels et pour conserver une configuration cohérente lorsque j'utiliserai mon Raspberry Pi pour d'autres projets, notamment en programmation, en électronique et en développement Linux.

Pour les laboratoires, je fournirai les identifiants nécessaires à ma professeure si elle doit accéder à mon Raspberry Pi. Je devrai également tenir compte de cette différence dans les commandes et les chemins de fichiers, puisque mon répertoire personnel est `/home/matisserv` plutôt que `/home/etudiant`.

Cette décision me permet de conserver mon Raspberry Pi comme environnement de développement personnel tout en l'utilisant pour mes cours.

Note de découverte : Même lorsque je suis connecté à distance avec mon MacBook, le Raspberry Pi utilise les raccourcis de copier-coller de Linux dans son bureau, plutôt que ceux de macOS.

https ://www.raspberrypi.com/software/operating-systems/

https ://www.raspberrypi.com/documentation/computers/software-sources.html

https ://forums.raspberrypi.com/viewtopic.php ?t=395293

17. Découverte physique du Raspberry Pi 5

Le laboratoire demandait aussi d'ouvrir le boîtier et de reconnaître les éléments comparables à ceux d'un ordinateur classique. C'est une partie importante parce qu'avant de faire mes essais de réseau, je voulais aussi savoir ce que

j'avais réellement entre les mains.

Le Raspberry Pi 5 n'a pas la même présentation qu'un ordinateur de bureau. Son processeur fait partie du **SoC Broadcom BCM2712**, la mémoire vive est soudée sur la carte et le système peut démarrer depuis une carte microSD. Il possède également des connecteurs USB, Ethernet, micro-HDMI, USB-C, ainsi qu'un connecteur GPIO à 40 broches pour les projets électroniques.

Élément que je voulais identifier	Son équivalent ou son rôle
SoC BCM2712	Regroupe notamment le processeur principal
RAM	Mémoire temporaire utilisée par les programmes
Carte microSD	Stockage du système et des fichiers dans mon montage
RP1	Contrôleur d'une partie des entrées-sorties
Ports USB et Ethernet	Connexions aux périphériques et au réseau
Micro-HDMI	Sortie vidéo vers un écran
GPIO 40 broches	Liaisons avec des composants électroniques
Connecteur USB-C	Alimentation du Raspberry Pi

J'ai aussi pu constater la différence entre mes premiers essais sans écran et une utilisation avec un écran physique connecté en micro-HDMI. Dans le deuxième cas, je pouvais vérifier directement le terminal, ce qui m'a été utile pour poursuivre mes recherches sur l'inventaire des Pi.

Photos à joindre : je veux encore ajouter des photos de ma vraie carte et les annoter pour identifier les composants et les connecteurs. Je préfère conserver cet élément comme travail à terminer plutôt que d'insérer une photographie générique en laissant croire qu'il s'agit de mon montage. Les caractéristiques techniques peuvent être vérifiées dans la documentation officielle du Raspberry Pi 5.

18. Python et essais supplémentaires

Une autre partie du laboratoire portait sur la découverte de Python. J'avais déjà commencé à explorer ce langage de mon côté, mais je voulais surtout comprendre comment l'utiliser directement sur un Raspberry Pi pour faire autre chose que de simples exemples dans le terminal.

18.1 Mon idée : récupérer des informations sur les Pi utilisés en classe

Pendant mes essais, j'ai voulu trouver une manière de récupérer et de conserver des informations sur les Raspberry Pi, par exemple leur nom d'hôte et certaines informations système, pour pouvoir les distinguer. Au départ, j'ai envisagé un inventaire où les informations seraient associées aux appareils du magasin et aux postes utilisés en classe.

J'ai aussi utilisé un câble micro-HDMI pour accéder directement à l'affichage d'un Pi, parce que c'était plus pratique à ce moment-là que de continuer à chercher l'appareil uniquement par le réseau.

18.2 Un premier essai d'enregistrement dans SQLite

J'ai travaillé sur une solution avec un Pi serveur, un script Python et une base de données **SQLite**. Le principe était qu'un Pi client envoie des informations au serveur, puis que ces informations soient conservées pour pouvoir les consulter plus tard.

Pendant nos essais, **un Raspberry Pi étudiant a réellement exécuté la commande prévue et son inscription a été enregistrée dans la base SQLite du Pi serveur**. J'ai ensuite pu retrouver cette entrée avec la commande :

```
python3 registry.py --list
```

Ce résultat confirmait qu'au moins une inscription avait fonctionné. Par contre, je ne veux pas confondre **cet essai réussi avec un Pi** et **l'inventaire complet de tous les appareils de la classe**. Je n'ai pas encore de preuves montrant que le système complet a été déployé et validé pour chacun d'eux.

L'architecture de démarrage automatique, les vérifications réseau supplémentaires, la sauvegarde complète et le tableau de bord sont des éléments sur lesquels j'ai aussi travaillé, mais ils demandent des validations distinctes.

18.3 Ce que j'ai appris avec cette expérience

Cet essai m'a permis de faire un lien concret entre plusieurs sujets : les scripts Python, la communication réseau, les informations système de Linux et l'enregistrement dans une base de données. J'ai aussi appris qu'un programme qui fonctionne sur un appareil ne prouve pas automatiquement qu'il sera compatible avec tous les autres.

Je veux encore joindre le code source correspondant à la version réellement testée, les commandes utilisées et des captures des résultats qui ne contiennent pas d'informations personnelles d'autres étudiants. Je n'ajouterai pas de résultats que je n'ai pas pu vérifier.

19. Sources et références

[1] Raspberry Pi Ltd. — Getting started

Installation du système, configuration d'une microSD, SSH et utilisation sans écran.

https ://www.raspberrypi.com/documentation/computers/getting-started.html

[2] Raspberry Pi Ltd. — Remote access

Documentation officielle de SSH, VNC, TigerVNC et SCP.

https ://www.raspberrypi.com/documentation/computers/remote-access.html

[3] Raspberry Pi Ltd. — Configuration

Documentation du réseau, du Wi-Fi, de NetworkManager et de Raspberry Pi OS.

https ://www.raspberrypi.com/documentation/computers/configuration.html

[4] Raspberry Pi Ltd. — Raspberry Pi Connect

Service d'accès à distance évalué comme solution possible au début du dépannage.

https ://www.raspberrypi.com/documentation/services/connect.html

[5] Apple — À propos de networksetup

Configuration des paramètres réseau macOS en ligne de commande.

https ://support.apple.com/fr-ca/guide/remote-desktop/apdd0c5a2d5/mac

[6] dnsmasq — Documentation officielle

Description du serveur DHCP et des paramètres `--interface`, `--dhcp-range`, `--dhcp-host` et `--dhcp-option`.

https ://thekelleys.org.uk/dnsmasq/docs/dnsmasq-man.html

[7] NetworkManager — Documentation nmcli

Gestion des interfaces et des connexions réseau Linux.

https ://networkmanager.pages.freedesktop.org/NetworkManager/NetworkManager/nmcli.html

[8] NetworkManager — Référence des paramètres

Paramètres IPv4, routes et authentification IEEE 802.1X.

https ://networkmanager.pages.freedesktop.org/NetworkManager/NetworkManager/nm-settings-nmcli.html

[9] Cégep Limoilou — Direction des ressources informationnelles

Connexion Wi-Fi Android et iPhone : paramètres eduroam, PEAP, MSCHAPV2 et certificats. Ces paramètres institutionnels ont été adaptés à Linux.

https ://www.cegeplimoilou.ca/app/uploads/2025/06/Connexion-wi-fi-pour-android-et-iphone.pdf

[10] Homebrew — Documentation officielle

Gestion et installation de logiciels sur macOS.

https ://docs.brew.sh/

[11] TigerVNC — Site officiel

Client VNC utilisé pour afficher et contrôler le bureau du Raspberry Pi.

https ://tigervnc.org/

[12] RFC 2131 — Dynamic Host Configuration Protocol

Fonctionnement et échanges du protocole DHCP.

https ://www.rfc-editor.org/rfc/rfc2131

[13] RFC 3927 — Dynamic Configuration of IPv4 Link-Local Addresses

Adresses IPv4 automatiques dans le bloc 169.254.0.0/16.

https ://www.rfc-editor.org/rfc/rfc3927

20. Journal complémentaire de mes expériences et de mes recherches sur les Raspberry Pi 5

Dans le rapport initial, j'avais surtout écrit ce que j'avais réussi à faire le 27 octobre : le réseau, SSH, VNC et eduroam. Mais depuis, j'ai continué de travailler avec les Raspberry Pi, autant au cégep qu'à la maison. J'ai aussi essayé de comprendre comment les utiliser pour faire tourner des modèles d'intelligence artificielle et pour communiquer entre plusieurs appareils.

Je voulais ajouter ces informations sans faire croire que j'avais déjà tout installé ou tout testé. Je vais donc utiliser trois mentions dans cette partie :

- **Réalisé et observé :** j'ai exécuté la manipulation et j'ai un résultat à rapporter.
- **Tenté, mais non résolu :** j'ai vraiment fait un essai, mais je n'ai pas obtenu le résultat voulu ou je n'ai pas identifié la cause.
- **Recherché, préparé ou prévu :** j'ai travaillé sur l'idée, les scripts ou la documentation, mais je n'ai pas encore de validation sur le matériel réel.

20.1 Les Raspberry Pi 5 dont j'ai parlé ne sont pas tous le même appareil

C'est une distinction importante parce que sinon les résultats peuvent avoir l'air contradictoires. Au laboratoire, j'ai utilisé un Raspberry Pi mis à ma disposition. Ensuite, j'ai eu mon Raspberry Pi personnel. J'ai aussi travaillé sur des idées qui concernent les Raspberry Pi du magasin TGE.

Appareil	Ce que je sais	Situation
Pi 5 utilisé au laboratoire	Pi du laboratoire, installé sur une grande carte ou base verte ; quantité de RAM non confirmée	J'ai fait mes manipulations réseau du 27 octobre
Pi 5 personnel	Starter Kit 8 Go de PiShop Canada, référence K2B-1345-8GB	Assemblé et démarré ; mesures système réalisées
Pi 5 du magasin identifié « Excel »	Présenté comme un appareil de 8 Go ; à revérifier sur l'appareil	Associé à mes démarches d'inventaire et aux projets d'équipe
Autres Pi 5 du magasin	Modèles et numéros de référence à inventorier	Je voulais récupérer leurs adresses MAC et les identifier

Les adresses IP et les valeurs du laboratoire ne doivent donc pas être attribuées automatiquement à mon Pi personnel.

20.2 Ce que j'ai réussi le 27 octobre : un vrai réseau direct entre mon Mac et le Pi

Je ne vais pas recopier toutes les commandes ici, parce qu'elles sont déjà dans les sections 4 à 14. Par contre, c'est quand même une des expériences les plus intéressantes que j'ai faites avec le Raspberry Pi.

Au début, mon Mac reconnaissait la liaison Ethernet, mais ce n'était pas suffisant pour communiquer. Je voyais une adresse automatique `169.254.30.16` et je n'arrivais pas à résoudre `etudiant.local`. J'ai utilisé `tcpdump` et je suis tombé sur les requêtes DHCP du Raspberry Pi. C'est à ce moment-là que j'ai compris qu'un câble actif n'implique pas nécessairement une bonne configuration IP.

Ensuite, j'ai installé `dnsmasq` sur mon Mac, configuré une petite plage DHCP et obtenu l'adresse `192.168.77.2` pour le Pi. J'ai pu me connecter en SSH et afficher son bureau avec TigerVNC. Finalement, j'ai mis des adresses IP fixes pour ne plus avoir besoin de laisser `dnsmasq` actif.

J'ai aussi réussi à faire une chose que je voulais vraiment : utiliser l'Ethernet pour mon lien Mac–Pi et le Wi-Fi `eduroam` pour Internet. La commande `ip route get 1.1.1.1` montrait que le Pi passait par `wlan0`, donc les deux connexions ne se remplaçaient pas.

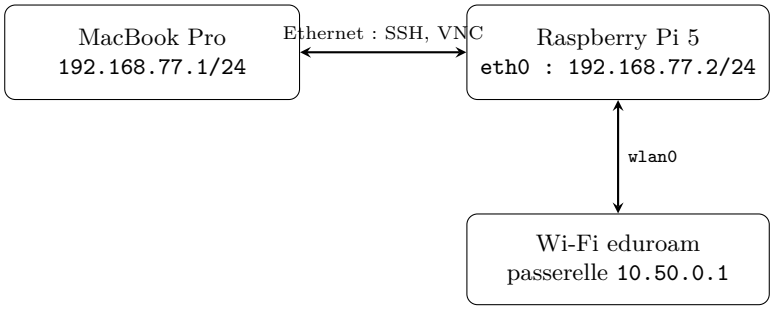


FIGURE 1 – Schéma de la configuration réseau que j'ai réellement obtenue le 27 octobre 2026.

20.3 Le 2 novembre : pourquoi j'ai gardé aussi les échecs dans le rapport

Quand j'ai voulu refaire la connexion au cégep, le `ping` vers `192.168.77.2` n'a pas répondu. L'entrée ARP était `(incomplete)` et SSH indiquait `Host is down`.

Je n'ai pas trouvé une cause définitive pendant cet essai. Je préfère le noter plutôt que de dire que ma solution était parfaitement stable. Ça m'a appris qu'une configuration qui fonctionne une journée n'est pas automatiquement reproductible si je change d'adaptateur, de port Ethernet, d'alimentation, de Pi ou de contexte réseau.

Les tests à refaire sont : vérifier le lien Ethernet, le nom de l'interface macOS, les adresses configurées sur les deux appareils, les voisins ARP et, si possible, une capture `tcpdump` au moment du démarrage.

20.4 Le Raspberry Pi 5 personnel : assemblage, démarrage et choix de l'accès à distance

J'ai reçu un **Raspberry Pi 5 Starter Kit 8 Go** et j'ai suivi le manuel fourni pour le préparer. J'ai réussi à obtenir son bureau sur un écran physique. Ensuite, comme je veux surtout travailler avec mon MacBook, j'ai essayé de récupérer ce bureau à distance.

J'ai d'abord activé **Raspberry Pi Connect**. J'ai réussi à utiliser l'accès à distance, mais j'ai eu des ralentissements, des images figées et parfois un écran noir. J'ai donc essayé **TigerVNC** sur mon Mac. Avec l'adresse locale `10.0.0.57`, j'ai réussi à accéder au bureau du Pi. Dans mon cas, c'était une solution plus pratique pour continuer à travailler.

Je ne prétends pas que TigerVNC sera toujours plus rapide que Raspberry Pi Connect : c'est mon observation pendant mes essais, dans mon réseau et avec mon matériel.

20.5 Vérification de la version de Linux installée

Sur mon Pi personnel, je n'ai pas voulu réinstaller le système juste pour le réinstaller. J'ai commencé par vérifier ce qui était déjà présent avec `cat /etc/os-release`, `uname -m`, `getconf LONG_BIT` et `uname -r`.

Vérification	Résultat observé
Distribution	Debian GNU/Linux 13
Version	13.7
Nom de code	Trixie
Architecture retournée par `uname -m`	`aarch64`
Taille de l'architecture	64 bits
Noyau	`6.18.50+rpt-rpi-2712`

J'ai donc décidé de conserver ce système pour mes essais. Je fais quand même une différence entre **identifier la base Debian et le noyau** et **prouver de quelle image exacte provenait une carte microSD préinstallée** : les commandes précédentes ne donnent pas nécessairement toute cette information.

20.6 Mes premières vraies mesures de ressources système

Le 1er novembre, j'ai aussi voulu voir dans quel état était mon Pi 5 personnel. Je voulais savoir si le système chauffait, si le processeur était limité et combien de RAM Linux utilisait avant même de lancer un modèle d'IA.

Mes commandes étaient notamment :

```
vcgencmd get_throttled
vcgencmd measure_temp
free -h
uptime
```

J'avais obtenu les résultats suivants :

Indicateur	Valeur réellement relevée	Ce que j'en conclus
État de limitation `get_throttled`	`throttled=0x0`	Aucun indicateur de limitation signalé au moment du relevé
Température	**35,6 °C**	Température faible pendant cet état de fonctionnement
RAM totale affichée	**7,9 Gio**	Cohérent avec mon modèle 8 Go
RAM utilisée	**474 Mio**	Faible utilisation dans cet état du système
RAM disponible	**environ 7,4 Gio**	Mémoire encore disponible pour les programmes
Swap utilisée	**0 octet**	Pas d'utilisation de swap dans ce relevé
Charges 1 / 5 / 15 minutes	**0,07 / 0,09 / 0,07**	Pi très peu sollicité
Temps depuis le démarrage	**environ 7 minutes**	Relevé pris peu après le démarrage

Petit calcul que j'ai voulu faire avec mes propres résultats : pour estimer la proportion de RAM utilisée, je convertis 7,9 Gio en Mio :

$$7{,}9 \times 1024 = 8089{,}6 \text{ Mio.}$$

Ensuite :

$$\frac{474}{8089{,}6} \times 100 \approx 5{,}86\ \%.$$

Donc, au moment de cet essai, la mémoire utilisée représentait **environ 5,9 %** de la RAM affichée. Il s'agit d'une estimation à partir des valeurs arrondies de `free -h`, pas d'une mesure plus précise que les données d'origine.

Pour interpréter la charge Linux, j'ai aussi appris qu'une moyenne de charge n'est pas la même chose qu'un pourcentage d'utilisation CPU. Une valeur de 0,07 ne signifie pas directement « 7 % du processeur ».

Enfin, `throttled=0x0` ne prouve pas que le Pi ne surchauffera jamais. Il faudrait refaire les mesures pendant une charge soutenue, en vérifiant la fréquence et les indicateurs de limitation.

20.7 Mon idée de récupérer les adresses MAC des Pi du magasin TGE

J'ai voulu trouver une façon de faire un inventaire des Raspberry Pi 5 disponibles au magasin. Mon idée était de brancher un Pi, de récupérer son adresse MAC et son adresse IP lorsque possible, puis d'associer ces informations à sa référence de magasin.

J'avais pensé utiliser mon Mac, son adaptateur Ethernet et une capture réseau. Mais il y avait une difficulté importante : **certains Raspberry Pi n'avaient pas de carte microSD ou de système d'exploitation disponible**. Dans ces conditions, je ne peux pas présumer qu'un Pi va démarrer un client DHCP ou envoyer une trame Ethernet exploitable. Une adresse MAC ne sera pas nécessairement visible simplement parce qu'un câble est branché.

J'ai donc considéré des méthodes avec un système de démarrage temporaire, un accès au terminal ou des outils USB. À cette étape, il s'agissait surtout de préparer une méthode d'inventaire, pas d'un inventaire complet déjà validé.

20.8 Tests envisagés entre deux Raspberry Pi 5

Une autre idée était de ne plus utiliser mon Mac comme intermédiaire, mais de faire communiquer directement deux Raspberry Pi 5 :

— Mon Pi personnel, utilisé comme serveur.
— Un Pi du magasin, utilisé comme client.

Le Pi du magasin a pu démarrer et me donner accès au terminal lors de la préparation. J'avais envisagé une liaison Ethernet directe et l'utilisation d'une clé USB pour transférer des scripts.

Une configuration d'essai a été proposée avec `10.77.55.1` pour le serveur et `10.77.55.2` pour le client. Ces adresses appartiennent à un **plan de configuration proposé**, pas à un relevé de connexion réelle confirmé entre les deux Pi.

Des scripts et une petite logique SQLite d'inventaire ont aussi été préparés et testés dans un environnement local de développement. Ce n'est **pas la même chose** qu'avoir validé le tout sur les deux Raspberry Pi physiques. La prochaine preuve à conserver serait un `ping`, une connexion SSH ou une transaction client–serveur exécutée sur les appareils concernés.

20.9 Le serveur que je voudrais pouvoir démarrer et arrêter quand je veux

Je me suis aussi intéressé à une méthode plus pratique pour mes futurs laboratoires : brancher mon ordinateur au réseau expérimental, lancer un script qui configure et démarre le serveur, puis le fermer proprement avec un autre script.

L'objectif était que l'arrêt du serveur sauvegarde les données, ferme les connexions et me dise clairement où se trouve la base de données. J'avais même envisagé un export sur le Bureau de mon ordinateur.

J'ai discuté de l'architecture, des scripts de démarrage et d'arrêt ainsi que de l'idée d'utiliser SQLite. Je n'ai cependant pas dans mes notes une démonstration complète attestant que tout ce cycle de démarrage, fonctionnement, arrêt et sauvegarde a déjà été exécuté avec succès sur le matériel réel. Je le conserve donc comme **travail préparatoire**.

---

21. Les notions techniques que j'ai voulu approfondir

> **Statut de cette extension :** les sections 1 à 19 ci-dessus reprennent le rapport de départ. Les sections suivantes sont un complément théorique, des calculs illustratifs et des protocoles d'expériences à réaliser.

Aucun résultat de performance non mesuré n'est présenté comme une observation personnelle. Les éléments incomplets des réalisations 1 et 3 restent explicitement à valider.

21.1 Correspondance avec les exigences du laboratoire

Exigence du laboratoire 08	Éléments présents dans ce rapport	Vérification restante
Réalisation 1 : composants et comparaison PC	Sections 2, 3, 17 et 21	Photographies annotées, inspection matérielle
Réalisation 2 : installation et réseau	Sections 3 à 16	Vérifier la reproductibilité du montage
Réalisation 3 : Python et outils	Sections 18, 23 et 29	Installer/exécuter et conserver les sorties
Bonis : scripts et photos	Protocoles, exemples et figures	Ajouter fichiers exécutés et photos réelles
Sources techniques	Références du rapport et section 31	Vérifier chaque référence utilisée

Je tiens à garder le laboratoire officiel au centre du rapport : il demande aussi de documenter le montage, les connecteurs, les différences avec un ordinateur conventionnel et les outils Python. Ces exigences sont prioritaires sur les approfondissements facultatifs en IA.

22. Architecture du Raspberry Pi 5 et comparaison avec un PC

22.1 Un ordinateur monocarte, pas un microcontrôleur

Le Raspberry Pi 5 est un *single-board computer* (SBC). Il exécute un système d'exploitation généraliste avec gestion de mémoire virtuelle, processus, fichiers, pilotes et réseau. Il ne doit pas être confondu avec un microcontrôleur STM32, généralement utilisé pour des tâches embarquées plus déterministes.

Le SoC Broadcom BCM2712 intègre quatre cœurs Arm Cortex-A76 à 2,4 GHz. La carte possède aussi le contrôleur d'entrées-sorties RP1. La mémoire vive est soudée sur la carte et le stockage de démarrage est souvent une carte microSD, contrairement à un disque dur traditionnel. La capacité RAM exacte de l'exemplaire utilisé doit être vérifiée plutôt que déduite du modèle de carte.

Élément	Raspberry Pi 5	Ordinateur de bureau classique
Processeur	SoC ARM avec CPU intégré	CPU x86-64 ou ARM, souvent sur support distinct
RAM	LPDDR soudée	DIMM/SO-DIMM ou mémoire intégrée
Stockage	microSD, USB ou SSD PCIe selon montage	SSD NVMe/SATA ou autre
Graphique	GPU intégré	GPU intégré ou carte dédiée
GPIO	Connecteur 40 broches	Pas toujours accessible
Alimentation	USB-C, alimentation adaptée	Bloc ATX ou alimentation propriétaire
Refroidissement	Passif ou ventilateur selon boîtier	Dissipateur et ventilation dimensionnés
Usage	Prototypage, Linux embarqué, réseau, IA légère	Applications générales et calcul intensif

La fréquence nominale ne suffit pas pour comparer deux processeurs : architecture, IPC, mémoire, refroidissement et logiciels influencent les performances.

22.2 Schéma matériel conceptuel

Le schéma suivant est une **figure vectorielle créée en LaTeX/TikZ**, intégrée directement au PDF sans image externe. Les flèches montrent des liens fonctionnels et non le routage électrique exact du PCB.

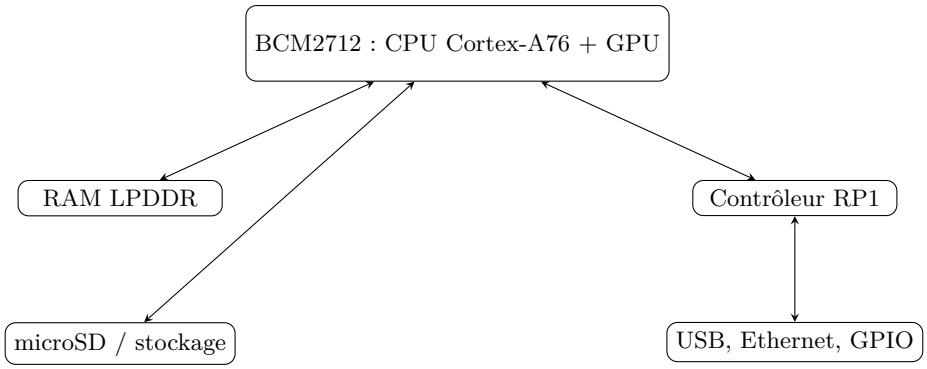


FIGURE 2 – Schéma fonctionnel simplifié du Raspberry Pi 5 (illustration conceptuelle).

22.3 Inventaire reproductible de l'appareil

Sur le Pi, les commandes suivantes permettent de consigner la configuration réelle :

```
cat /etc/os-release
uname -a
uname -m
lscpu
free -h
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
df -h
ip -br address
vcgencmd measure_temp
```

Conserver les sorties dans un dossier de preuves, en masquant les identifiants et informations sensibles. Ne jamais copier des mots de passe ou fichiers de configuration contenant des secrets dans un dépôt public.

23. Linux : du démarrage aux processus

23.1 Chaîne de démarrage

De manière simplifiée, le micrologiciel initialise le matériel, charge les composants nécessaires au démarrage et le noyau Linux prend le relais. Le noyau détecte les périphériques, initialise les pilotes et monte les systèmes de fichiers. Le système d'initialisation lance ensuite les services et éventuellement le bureau graphique.

Le noyau travaille en mode privilégié (*kernel space*), alors que les applications fonctionnent principalement en *user space*. Les appels système fournissent une interface contrôlée entre les deux.

```
systemctl --failed
systemctl status ssh
journalctl -b -p warning
ps aux --sort=-%mem | head
top
```

Les commandes ci-dessus servent à analyser les services, journaux et processus. Elles ne constituent pas des preuves de résultats tant que leurs sorties n'ont pas été conservées.

23.2 Mémoire virtuelle, cache et swap

Un programme utilise des adresses virtuelles traduites en adresses physiques par le système de gestion mémoire. Linux exploite aussi la RAM libre pour les caches de fichiers. Ainsi, une mémoire affichée comme « utilisée » n'est pas forcément indisponible pour les applications.

Le *swap* peut éviter certains échecs d'allocation, mais il ne remplace pas la RAM : sur microSD, il risque de provoquer une forte dégradation des performances et davantage d'écritures.

23.3 Diagnostic réseau par couches

Couche	Question	Outil
Physique / liaison	Le câble et le lien Ethernet sont-ils actifs ?	`ip link`, `ethtool`
Adressage	Quelle adresse IP est configurée ?	`ip addr`, `nmcli`
Routage	Quelle interface est utilisée ?	`ip route get 1.1.1.1`
Voisinage	L'adresse MAC est-elle résolue ?	`ip neigh`, `arp`
Transport	Le service écoute-t-il ?	`ss -lnt`
Application	SSH ou VNC répond-il ?	`ssh`, client VNC

Le rapport initial documente déjà un échec ARP et un succès DHCP. Il est important de ne pas conclure qu'une machine est éteinte simplement parce qu'un `ping` échoue.

24. Python sur Raspberry Pi OS

24.1 Installation et environnement virtuel

Sur une distribution Debian récente, l'utilisation d'un environnement virtuel évite de modifier les paquets Python gérés par le système.

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip
python3 --version
mkdir -p ~/lab08-python
cd ~/lab08-python
python3 -m venv .venv
source .venv/bin/activate
python -m pip --version
```

L'installation effective et la version obtenue doivent être ajoutées au journal d'expérience. Un environnement Conda peut aussi être étudié, mais il n'est pas obligatoire pour comprendre `venv`.

24.2 Premier programme mesurable

```
import os
import platform
import time

start = time.perf_counter()
total = sum(i * i for i in range(1_000_000))
elapsed = time.perf_counter() - start

print("Architecture :", platform.machine())
print("Coeurs logiques :", os.cpu_count())
print("Somme :", total)
print(f"Duree : {elapsed:.6f} s")
```

Ce test est un exercice Python et non un benchmark CPU standardisé. Les temps dépendent de l'interpréteur, de la température et des tâches concurrentes.

25. Introduction à l'intelligence artificielle locale

25.1 Différencier IA, apprentissage automatique et LLM

Avant d'essayer des modèles sur mon Pi, j'ai voulu comprendre ce qu'on appelait exactement un LLM, parce que je voyais plusieurs termes qui avaient l'air semblables sans être identiques. L'intelligence artificielle regroupe des méthodes permettant à un système d'accomplir certaines tâches associées à des capacités cognitives. L'apprentissage automatique construit des modèles à partir de données. L'apprentissage profond utilise notamment des réseaux neuronaux multicouches.

Un **LLM** (*large language model*) est un modèle statistique entraîné sur de grandes quantités de données linguistiques et, parfois, multimodales. Il prédit des unités de texte appelées *tokens*. Il n'effectue pas une recherche dans une base de réponses prédéfinies à chaque mot généré.

Un modèle de classification prédit une classe, par exemple « chat » ou « chien ». Un modèle de débruitage estime un signal plus propre à partir d'une observation bruitée. Un modèle de diffusion apprend à inverser progressivement un processus de bruitage. Ces modèles ne sont pas nécessairement des LLM.

25.2 Entraînement et inférence

Pendant l'entraînement, les paramètres du modèle sont ajustés pour réduire une fonction de perte. Pendant l'inférence, les paramètres sont généralement fixes : le modèle calcule une sortie à partir de l'entrée. Sur un Raspberry Pi 5, on vise surtout l'inférence de modèles compacts et quantifiés ; entraîner un grand LLM à partir de zéro n'est pas réaliste.

26. Comment un LLM génère du texte

26.1 Tokenisation

Le texte est transformé en une séquence d'identifiants entiers. Un token peut correspondre à un mot, une partie de mot, un caractère ou une séquence de symboles selon le tokeniseur. Les nombres de tokens varient d'un modèle à l'autre.

Soit la séquence :

$$x = (x_1, x_2, \dots, x_T).$$

Un modèle autorégressif estime la probabilité du prochain token conditionnellement aux précédents :

$$P(x_{T+1} \mid x_1, \dots, x_T).$$

La probabilité d'une séquence entière se factorise par la règle de la chaîne :

$$P(x_1, \dots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, \dots, x_{t-1}).$$

26.2 Embeddings

Chaque identifiant est associé à un vecteur appris de dimension d. Pour un vocabulaire de taille V, la matrice d'embeddings est :

$$E \in \mathbb{R}^{V \times d}.$$

Le modèle ajoute ou applique une information de position, car l'ordre des tokens est indispensable pour comprendre une phrase.

26.3 Attention causale

À partir des représentations d'entrée X, des projections linéaires produisent les requêtes, clés et valeurs :

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

L'attention mise à l'échelle s'écrit :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V.$$

Ici, d_k est la dimension des clés et M est un masque causal : les positions futures reçoivent une valeur négative très grande avant softmax, afin que le modèle ne les consulte pas.

La fonction softmax transforme des scores en probabilités :

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Exemple numérique : pour deux scores $z = (1, 2)$,

$$p_1 = \frac{e^1}{e^1 + e^2} \approx 0{,}26894, \qquad p_2 = \frac{e^2}{e^1 + e^2} \approx 0{,}73106.$$

La deuxième position reçoit donc environ 73,1 % du poids d'attention, avant combinaison des vecteurs de valeurs.

26.4 Attention multi-têtes et couches profondes

L'attention multi-têtes applique plusieurs projections en parallèle pour capter différentes relations. Une couche Transformer contient habituellement de l'attention, des connexions résiduelles, des normalisations et un réseau feed-forward. Des architectures modernes utilisent des variantes comme RMSNorm, RoPE, SwiGLU ou GQA. Les détails exacts dépendent du modèle.

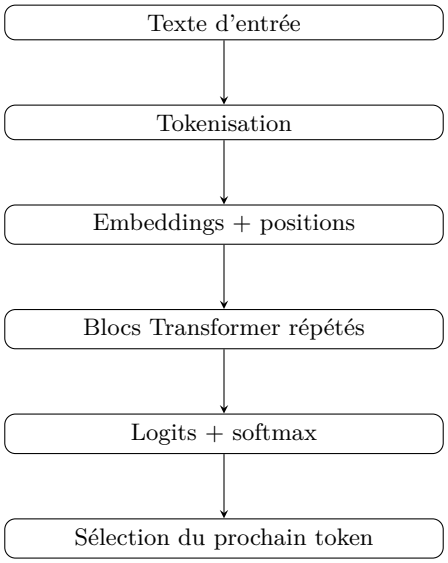


FIGURE 3 – Chaîne simplifiée de génération autorégressive d'un LLM.

26.5 Logits, température et échantillonnage

Les *logits* sont les scores non normalisés produits par le modèle. Avec une température $\tau > 0$:

$$P_i(\tau) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}.$$

Une température plus basse rend généralement la distribution plus concentrée. Une température élevée augmente la diversité, sans garantir une meilleure réponse. Les méthodes *top-k* et *top-p* restreignent les candidats avant échantillonnage.

27. Mathématiques de l'entraînement

27.1 Produit matriciel et paramètres

Pour une couche linéaire :

$$Y = XW + b.$$

Si $X \in \mathbb{R}^{m\times n}$ et $W \in \mathbb{R}^{n\times p}$, alors $Y \in \mathbb{R}^{m\times p}$. Le produit requiert approximativement $2mnp$ opérations arithmétiques si une multiplication et une addition sont comptées séparément.

27.2 Perte d'entropie croisée

Pour une cible de classe y et une probabilité prédite p_y :

$$\mathcal{L} = -\ln(p_y).$$

Si le modèle attribue $p_y = 0{,}8$ au bon token, la perte est :

$$\mathcal{L} = -\ln(0{,}8) \approx 0{,}22314.$$

Si $p_y = 0{,}1$, elle vaut environ 2,30259. Le modèle est davantage pénalisé lorsqu'il donne une faible probabilité à la bonne réponse.

27.3 Descente de gradient

Un paramètre θ peut être mis à jour selon :

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t),$$

où η est le taux d'apprentissage. La rétropropagation calcule les dérivées grâce à la règle de la chaîne. Les optimiseurs modernes, notamment AdamW, adaptent les mises à jour et utilisent des statistiques des gradients.

27.4 Perplexité

Pour une perte moyenne en logarithme naturel :

$$\mathrm{PPL} = \exp(\overline{\mathcal{L}}).$$

Une perte moyenne de 2 donne une perplexité d'environ 7,389. Les perplexités ne sont pas directement comparables entre tokeniseurs et jeux de données différents.

28. Quantification des LLM

28.1 Pourquoi réduire la précision ?

C'est cette partie qui m'intéresse particulièrement pour un Raspberry Pi 5 avec une quantité limitée de RAM : je veux comprendre ce qu'on gagne en réduisant les bits, mais aussi ce qu'on peut perdre. Les paramètres d'un modèle sont souvent stockés en virgule flottante pendant l'entraînement. Pour l'inférence, on peut réduire leur représentation à 8, 4 ou moins de bits, avec des métadonnées d'échelle. Cela diminue la mémoire occupée, mais peut introduire une erreur de quantification.

Pour N paramètres et b bits par paramètre, la taille idéale des poids est :

$$M_{\text{poids}} = \frac{Nb}{8} \text{ octets.}$$

Attention : cette formule ignore les échelles, les *zero-points*, les métadonnées, les tenseurs non quantifiés, le KV cache, les activations et les buffers d'exécution.

28.2 Exemple pour un modèle de 3 milliards de paramètres

Précision idéale	Calcul	Poids seuls
FP32	$3 \times 10^9 \times 4$	12,0 Go
FP16 / BF16	$3 \times 10^9 \times 2$	6,0 Go
INT8	$3 \times 10^9 \times 1$	3,0 Go
INT4	$3 \times 10^9 \times 0{,}5$	1,5 Go
INT2	$3 \times 10^9 \times 0{,}25$	0,75 Go

Il s'agit de **Go décimaux** (10^9 octets). En Gio, il faut diviser le nombre d'octets par 2^{30}. Par exemple, 1,5 Go correspondent à environ 1,397 Gio.

28.3 Quantification affine

Une quantification uniforme simplifiée utilise :

$$q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right),$$

où s est l'échelle et z le point zéro. La déquantification approximative est :

$$\hat{x} = s(q - z).$$

Exemple : $x = 0{,}37$, $s = 0{,}1$, $z = 0$ donnent $q = 4$ et $\hat{x} = 0{,}4$. L'erreur absolue est 0,03.

28.4 GGUF et types de quantification

GGUF est un format de stockage de modèles et de métadonnées utilisé notamment dans l'écosystème llama.cpp. Des désignations comme Q4_K_M, Q5_K_M ou Q8_0 décrivent des méthodes de quantification particulières : elles ne signifient pas que **tous** les poids occupent exactement 4, 5 ou 8 bits.

La taille du fichier GGUF, la RAM effectivement utilisée et la qualité de sortie doivent être mesurées séparément. Une quantification plus agressive ne garantit pas toujours une accélération proportionnelle sur CPU.

29. Mémoire, KV cache et performances du Raspberry Pi 5

29.1 Estimation du KV cache

Pour un Transformer utilisant un cache clé-valeur, une approximation de la mémoire est :

$$M_{\mathrm{KV}} = 2\,L\,T\,H_{\mathrm{KV}}\,d_h\,b,$$

où L est le nombre de couches, T le nombre de tokens conservés, H_{KV} le nombre de têtes KV, d_h la dimension d'une tête et b le nombre d'octets par élément.

Exemple purement hypothétique : $L = 24$, $T = 2048$, $H_{\mathrm{KV}} = 8$, $d_h = 128$ et $b = 2$ octets :

$$M_{\mathrm{KV}} = 2(24)(2048)(8)(128)(2) = 201\,326\,592 \text{ octets} = 192 \text{ Mio.}$$

Le cache croît linéairement avec la longueur de contexte dans cette approximation. L'architecture et les optimisations réelles peuvent modifier le calcul.

29.2 Mémoire totale approximative

$$M_{\mathrm{total}} \approx M_{\mathrm{poids}} + M_{\mathrm{KV}} + M_{\mathrm{activations}} + M_{\mathrm{runtime}} + M_{\mathrm{OS}}.$$

Un fichier de poids de 3 Go ne signifie donc pas qu'un Pi disposant de 4 Go de RAM pourra nécessairement l'exécuter convenablement.

29.3 Débit théorique limité par la mémoire

Si un modèle doit lire approximativement S octets de poids par token et que la bande passante mémoire réellement disponible est B octets/s, une borne supérieure simplifiée est :

$$R_{\max} \lesssim \frac{B}{S} \quad \text{tokens/s.}$$

Exemple fictif, non mesuré sur Pi 5 : $B = 10$ Go/s et $S = 2$ Go donnent au plus environ 5 tokens/s dans un scénario strictement limité par la lecture des poids. La performance réelle peut être plus faible, et la réutilisation des données, les caches, le calcul et l'architecture changent cette approximation.

29.4 Temps de génération

Si le débit est r tokens/s et que n tokens sont générés :

$$t_{\text{génération}} \approx \frac{n}{r}.$$

Pour $n = 120$ et $r = 6$, on obtient $t \approx 20$ s, sans compter le traitement du prompt ni les surcoûts de lancement. Ces valeurs servent uniquement d'exemple numérique.

30. Protocole de benchmark reproductible des modèles locaux

30.1 Objectif

Mon objectif pour mes prochains essais est de comparer plusieurs modèles et plusieurs quantifications **sur le même Raspberry Pi**, avec la même longueur de contexte, le même prompt et des conditions thermiques comparables.

30.2 Informations à relever

Variable	Unité / format
Nom exact et version du modèle	texte
Architecture et nombre de paramètres	texte / milliards
Format et quantification	GGUF, Q4_K_M, etc.
Taille du fichier	Mio ou Gio
RAM installée et disponible	Gio
Nombre de threads	entier
Contexte configuré	tokens
Tokens du prompt et générés	tokens
Temps de préremplissage	secondes
Temps de génération	secondes
Débit de génération	tokens/s
Pic de mémoire RSS	Mio
Température CPU avant/après	°C
Éventuel throttling	oui/non
Qualité de réponse	protocole défini

30.3 Exemple de procédure avec llama.cpp

Après installation et compilation de llama.cpp selon sa documentation officielle, l'outil `llama-cli` peut servir à réaliser des essais. Les options précises doivent être vérifiées avec `--help` pour la version utilisée.

```
./build/bin/llama-cli \
  -m ./modeles/modele.gguf \
  -p "Explique la difference entre RAM et stockage." \
  -n 128 \
  -c 2048 \
  -t 4
```

Le chemin `./modeles/modele.gguf` est un exemple : aucun modèle n'est téléchargé par cette commande. Pour des mesures comparables, fixer la graine lorsque disponible, documenter les paramètres d'échantillonnage, répéter les essais et distinguer préremplissage et génération.

30.4 Tableau de résultats à remplir après exécution

Modèle	Quantification	RAM max (Mio)	Débit (tokens/s)	Température max (°C)
À mesurer	À mesurer	—	—	—
À mesurer	À mesurer	—	—	—
À mesurer	À mesurer	—	—	—

Aucun débit ni modèle n'est inventé ici. Ce tableau deviendra une preuve seulement après des essais documentés.

31. Classification, débruitage et modèles de diffusion

31.1 Classification

Un classificateur reçoit un vecteur ou une image et produit des scores de classes. Une fonction softmax peut transformer ces scores en probabilités. La classe prédite est souvent :

$$\hat{y} = \arg\max_k P(y = k \mid x).$$

La précision (*accuracy*) est :

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Pour une classe donnée, la précision positive et le rappel sont :

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}.$$

Le score F1 est la moyenne harmonique :

$$F_1 = 2\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Exemple : avec $TP = 40$, $FP = 10$ et $FN = 5$, on obtient une précision de 0,8, un rappel d'environ 0,8889 et un score F1 d'environ 0,8421.

31.2 Débruitage de signaux et d'images

Un modèle de débruitage cherche à reconstruire un signal x à partir d'une observation :

$$y = x + \varepsilon,$$

où ε représente le bruit. Dans un exemple gaussien, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$.

On peut entraîner un réseau f_θ à minimiser l'erreur quadratique moyenne :

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N} \left(f_\theta(y_i) - x_i\right)^2.$$

Pour deux pixels de référence $(100, 150)$ et deux prédictions $(110, 140)$, la MSE est :

$$\frac{(110-100)^2 + (140-150)^2}{2} = 100.$$

Le résultat est exprimé en unités d'intensité au carré. Pour des images 8 bits, le PSNR peut être calculé par :

$$\text{PSNR} = 10\log_{10}\left(\frac{255^2}{\text{MSE}}\right).$$

Avec une MSE de 100, le PSNR vaut environ 28,13 dB.

31.3 Modèles de diffusion

Dans une formulation classique de diffusion, le processus direct ajoute progressivement du bruit :

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}x_{t-1}, \beta_t I\right).$$

En posant $\alpha_t = 1 - \beta_t$ et $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, on peut échantillonner directement :

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I).$$

Un réseau apprend à prédire le bruit ou une autre quantité permettant d'inverser progressivement le processus. Ce mécanisme est différent de la génération token par token d'un LLM autorégressif, même si certaines architectures réutilisent des Transformers.

31.4 Expériences possibles sur Pi 5

— Classifier quelques images avec un petit modèle préentraîné et relever temps, RAM et exactitude.
— Débruiter des images artificiellement bruitées et mesurer MSE/PSNR.
— Comparer un filtre médian, un filtre gaussien et un modèle appris.
— Comparer les besoins matériels d'un classificateur, d'un débruiteur et d'un LLM.

Ces expériences sont proposées, **pas déclarées réalisées**.

32. Limites, sécurité, sources complémentaires et travaux futurs

32.1 Sécurité des réseaux et des modèles

Une carte partagée entre étudiants ne doit pas conserver des identifiants eduroam personnels accessibles aux autres utilisateurs. Utiliser des comptes distincts, limiter les droits, éviter de publier les profils NetworkManager et vérifier les règles du réseau institutionnel. Ne jamais publier de jetons d'accès aux dépôts de modèles ou services d'IA.

Un modèle local n'est pas automatiquement fiable ni exempt de risques. Il peut halluciner, produire des erreurs, contenir des biais et traiter des données confidentielles. Le fait de l'exécuter hors ligne limite certaines transmissions, mais ne garantit pas la sécurité du système.

32.2 Ce qui reste à faire

1. Photographier la carte et annoter les composants et connecteurs réels, puis ajouter ces vraies photos dans `images/`.
2. Relever les caractéristiques exactes du Raspberry Pi utilisé.
3. Exécuter les programmes Python et joindre les sorties.
4. Identifier les modèles IA réellement téléchargés et testés.
5. Mesurer plusieurs quantifications avec le même protocole.
6. Réaliser et documenter les essais de classification et de débruitage.
7. Compléter les tableaux sans remplacer les mesures manquantes par des estimations.
8. Réessayer le lien Ethernet qui avait cessé de fonctionner le 2 novembre.
9. Réaliser un essai complet et chronométré sur un modèle LLM quantifié, puis conserver le nom exact et les logs.

32.3 Sources techniques complémentaires

— Raspberry Pi Ltd., *Raspberry Pi 5 product brief* : https ://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf
— Raspberry Pi Ltd., *Documentation officielle* : https ://www.raspberrypi.com/documentation/
— Python Software Foundation, *venv* : https ://docs.python.org/3/library/venv.html
— Vaswani et al. (2017), *Attention Is All You Need* : https ://arxiv.org/abs/1706.03762
— Devlin et al. (2018), *BERT* : https ://arxiv.org/abs/1810.04805
— Kaplan et al. (2020), *Scaling Laws for Neural Language Models* : https ://arxiv.org/abs/2001.08361
— Hoffmann et al. (2022), *Training Compute-Optimal Large Language Models* : https ://arxiv.org/abs/2203.15556
— Dettmers et al. (2022), *LLM.int8()* : https ://arxiv.org/abs/2208.07339
— Frantar et al. (2022), *GPTQ* : https ://arxiv.org/abs/2210.17323
— Lin et al. (2023), *AWQ* : https ://arxiv.org/abs/2306.00978
— llama.cpp, code et documentation : https ://github.com/ggml-org/llama.cpp
— GGUF, spécification et outils : https ://github.com/ggml-org/ggml
— Ho et al. (2020), *Denoising Diffusion Probabilistic Models* : https ://arxiv.org/abs/2006.11239
— scikit-learn, métriques de classification : https ://scikit-learn.org/stable/modules/model_evaluation.html

Méthodologie documentaire : mes notes de manipulations des 27 octobre, 1er, 2, 3 et 6 novembre 2026 servent aussi à reconstituer le journal de la section 20. Les faits issus du rapport d'origine sont conservés dans les sections précédentes ; les développements théoriques de cette extension sont des explications générales fondées sur les références ci-dessus. Les calculs chiffrés explicitement désignés comme exemples ne constituent pas des mesures sur le Raspberry Pi.

Note sur l'utilisation de l'intelligence artificielle

Je veux être transparent sur la façon dont j'ai travaillé. Les manipulations que je présente comme réalisées sont celles que j'ai effectuées ou dont j'ai observé le résultat. Les commandes, les erreurs et les essais réseau restent au centre du rapport, même quand une IA m'a aidé à mieux comprendre ce que je voyais.

Assistance de ChatGPT — Je m'en suis servi pour obtenir des explications, poser des questions, vérifier certaines commandes, relever des erreurs et organiser le rapport en Markdown compatible avec Pandoc. ChatGPT a également participé à la reformulation et à la rédaction de certains passages à partir de mes observations, de mes notes et de mes corrections. Ce n'est donc pas seulement un correcteur orthographique. J'ai demandé qu'on conserve ma façon d'expliquer, mes objectifs et les résultats qui correspondent réellement à mes essais.

Il y a aussi une partie de l'analyse des journaux réseau où j'ai utilisé un **modèle d'IA local distinct de ChatGPT**, accessible sur une infrastructure de collaboration. Je ne peux pas publier le nom du modèle, la configuration, les détails du traitement ni les traces associées à cette utilisation, parce qu'une entente de confidentialité s'y applique. Je préfère l'indiquer clairement plutôt que de donner de faux détails ou de laisser croire que ce modèle fonctionnait directement sur le Raspberry Pi 5.

Pourquoi j'ai voulu garder ces notes dans le rapport

Quand je demande à une IA de m'expliquer un concept, je ne considère pas sa réponse comme une preuve. J'essaie de comprendre l'explication, de comparer ce qu'elle me dit avec la documentation technique et de vérifier les résultats avec mes propres commandes. Par exemple, quand j'ai rencontré une adresse IPv4 en `/16`, j'ai demandé une explication et ensuite un petit quiz pour m'assurer de comprendre la différence avec les adresses en `/24` qu'on avait vues en classe.

J'utilise également **Antidote 12** pour corriger mon français. Je souhaite que mon rapport soit lisible sans perdre le côté personnel de ma démarche : c'est moi qui ai voulu essayer ces manipulations, qui ai rencontré les problèmes et qui ai pris les décisions pendant les essais.

Je pense qu'il faut être prudent avec les détecteurs automatiques de textes générés par IA. Ils peuvent se tromper, particulièrement lorsque le style d'un texte est très soigné ou que l'auteur n'est pas anglophone. Une étude sur les biais de ces détecteurs présente des limites de fiabilité. Si vous avez des questions sur une manipulation ou un résultat, je préfère pouvoir vous montrer mes démarches, mon historique de fichiers et expliquer ce que j'ai fait.

Référence complémentaire : OpenAI, *ChatGPT*, https ://chatgpt.com/ (outil de soutien à la recherche, aux explications et à la rédaction, utilisé lors de la préparation du rapport en octobre et novembre 2026).